



1

座席 3 (Seats 3)

Author : 平木 康傑

小課題 1,2,3

2 人組に与える座席 N 組の位置を総当たりし、それぞれにおいて VIP 客が得る 2 個の座席の位置を得ることで、あり得る割り当てすべてに対応する答えの候補が得られる。それらの最大値が答えとなる。

この解法の時間計算量は $O((2N+2)^N)$ である。

小課題 4

VIP 客が座る 2 個の座席の番号を、左から x, y とおく。「座席 x よりも左」「座席 x, y のあいだ」「座席 y よりも右」の 3 つの区間の長さは、順に $x-1, y-x-1, 2N+2-y$ となる。各区間をいくつかの 2 人組に過不足なく与えることから、これらの長さはいずれも偶数である必要があり、逆にすべて偶数であれば割り当てを達成できる。

上の条件は、「 $1 \leq x < y \leq 2N$ 」「 x は奇数」「 y は偶数」をすべて満たすこととして言い換えられる。

よって、上を満たす整数組 (x, y) を総当たりし、それらにおける $A_x + A_y$ の最大値を求めればよい。

この解法の時間計算量は $\Theta(N^2)$ である。

満点解法

小課題 4 の解法において、整数 $y = 2, 4, \dots, 2N+2$ を昇順に見ることを考える。

y の値を固定したとき、 y より小さい奇数 x に対する A_x の最大値 $m_y = \max\{A_1, A_3, \dots, A_{y-1}\}$ をおく。このとき、すべての y における $m_y + A_y$ の最大値が答えになる。

$y = 2$ のとき $m_y = A_1$ である。 $y \geq 4$ において、 m_y, m_{y-2} の定義を見比べると、 $m_y = \max(m_{y-2}, A_{y-1})$ と簡潔に求まることがわかる。

したがって、整数 $y = 2, 4, \dots, 2N+2$ を昇順に見て、上の式によって m_y を得ることで、 $m_y + A_y$ の最大値によって答えが求まる。

この解法の時間計算量は $\Theta(N)$ である。



2

宝石商 (Jeweler)

Author : 増田 拓真

小課題 1

客 i が $S_j - 0.1$ から $T_j + 0.1$ の間に店を訪れることができない条件を考えると, $R_i < S_j$ または $T_j < L_i$ である. つまり客 i が店 j を訪れることができる条件は $S_j \leq R_i$ かつ $L_i \leq T_j$ となる. よって案 j についての答えは, $1 \leq i \leq N$ で $S_j \leq R_i$ かつ $L_i \leq T_j$ を満たす C_i の和として計算できる. 時間計算量は $\Theta(NM)$ である.

小課題 2

$S_j = T_j$ であるので, 問題は区間 $[L_i, T_i]$ に点 S_j が含まれるかどうかを判定する問題に帰着される. よって時刻 $1, 2, \dots, \max_j T_j$ について, その点が含まれる区間の C_i の和を前計算しておき, その値を出力すればよい. 具体的には, それぞれの客 i について $[L_i, R_i]$ に C_i を加えると解釈すると, これはいわゆる imos 法を用いて計算できる. 時間計算量は $\Theta(N + M + \max_j T_j)$ である.

小課題 3

$1 = S_j \leq R_i$ は常に成り立つため, $L_i \leq T_j$ のみを考えればよい. これは単純に累積和を用いて, 1 から $\max_j T_j$ までの各点について L_i がその点以下であるような C_i の和を前計算しておき, その値を出力すればよい. 時間計算量は $\Theta(N + M + \max_j T_j)$ である.

小課題 4, 満点

条件 $S_j \leq R_i$ かつ $L_i \leq T_j$ は, $R_i < S_j$ ではないかつ $L_i \leq T_j$ と同値である. ここで $R_i < S_j$ ならば, $L_i < R_i$, $S_j \leq T_j$ より $L_i \leq T_j$ も成り立つ. よって答えは「 $L_i \leq T_j$ を満たす C_i の和」から「 $R_i < S_j$ を満たす C_i の和」を引いたものと一致する.

「 $L_i \leq T_j$ を満たす C_i の和」, 「 $R_i < S_j$ を満たす C_i の和」はそれぞれ小課題 3 と同様にして累積和を用いて前計算できる. 時間計算量は $\Theta(N + M + \max_j T_j)$ である.



3

衣服 (Clothes)

Author : 林 涼太郎

小課題 1

以下のように場合分けを行う.

- $A_1 > 23$ の場合, 体感温度は 23 度にできない.
- $A_1 = 23$ の場合, 服を 1 枚も買うことなく体感温度を 23 度にできる.
- $A_1 < 23$ の場合, 種類 $23 - A_1$ の服を 1 枚買えば体感温度を 23 度にできる.

小課題 2

$A_N > 23$ の場合, 目標は達成不可能である. 以下のすべての小課題の解説において, $A_N \leq 23$ であるという仮定をおく. $B_i := 23 - A_i$ により $B_1, \dots, B_N$ を定めると, $B_1 > B_2 > \dots > B_N \geq 0$ となる.

この問題は次のように形式的に言い換えられる:

すべての要素が 1 以上 100 以下の整数であるような多重集合 $S = \{s_1, s_2, \dots, s_k\}$ であって, 「すべての $1 \leq i \leq N$ に対し, 総和が B_i になるような S の部分集合が存在する」という条件を満たすものを考える. S の要素数の最小値を求め, また最小値を達成する S の例を一つ構築せよ.

$N \leq 3$ の場合は, 以下のように場合分けを行うことで最適解が得られる.

- $N = 1$ である場合, $B_1 = 0$ ならば $S = \{\}$ とし, そうでなければ $S = \{B_1\}$ とする.
- $N = 2$ である場合, $B_2 = 0$ ならば $S = \{B_1\}$ とし, そうでなければ $S = \{B_1, B_2\}$ とする.
- $N = 3$ である場合,
 - $B_3 = 0$ ならば $S = \{B_1, B_2\}$ とする.
 - $B_3 > 0$ かつ $B_1 = B_2 + B_3$ ならば, $S = \{B_2, B_3\}$ とする.
 - 以上のいずれでもない場合, $S = \{B_1, B_2, B_3\}$ とする.



小課題 3

この小課題においては、 $B_1 = N - 1, B_2 = N - 2, \dots, B_N = 0$ である。

購入する服の枚数の最小値が k であるとする。このとき、 S の部分集合は高々 2^k 個しかないので、 S の部分集合の総和としてあり得る値も高々 2^k 個しかない。したがって、答えの下界として $2^k \geq N$ が得られる。

逆に、整数 k が $2^k \geq N$ を満たすとき、 $S = \{2^0, 2^1, \dots, 2^{k-1}\}$ とすることで、部分集合の総和として $0, 1, \dots, 2^k - 1$ の 2^k 通りを作ることができる。この小課題においては、 B_i の分布が特殊であるために、この構築により最適解が達成できる。

小課題 4, 5

整数 $B_1 + 1 \leq i \leq 100$ について、種類 i の種類を購入すると明らかに最適解を与えない。また、 $1 \leq i \leq B_1$ については、種類 i の服は高々 1 枚までしか購入しないとしてよい。これは、「購入した服の中に種類 i の服が 2 枚存在すれば、片方を種類 $2i$ の服に置き換える」という操作を行っても、 S のある部分集合の総和として作れなくなる整数が存在しないことから証明できる。

以上の考察により、 $s_1, s_2, \dots, s_k$ の中に重複はなく、かつ $\{s_1, s_2, \dots, s_k\}$ は $\{0, 1, \dots, B_1\}$ の部分集合になっているような $s_1, s_2, \dots, s_k$ の選び方のみ考えればよいとわかる。

$L = B_1$ とする。服の購入の仕方として 2^L 通りのすべてを試すことにする。購入する服の種類 $s_1, s_2, \dots, s_k$ を固定したときに、この部分集合の和として $B_1, B_2, \dots, B_N$ をすべて作ることができるかどうかを判定したい。

さらに 2^k 通りの部分集合をすべて試すことにすると、全体の時間計算量は $O(3^L + 2^L N)$ となり、小課題 4 に正答できる。

これを高速化するために部分和问题の要領で DP を用いて計算すると、小課題 5 にも正答できる。

小課題 6, 7

$S = \{1, 2, 4, 8, 16, 32\}$ を選ぶことで部分和として $0, 1, \dots, 63$ をすべて作ることができる。したがって、最適解において S のサイズは 6 以下である。

小課題 6 においては $31 \geq B_1 > B_2 > \dots > B_N \geq 0$ である。 $\{1, 2, \dots, 31\}$ の部分集合であってサイズ 6 以下のものは 942649 個存在する。これらすべてを列挙し、DP で有効な解かどうか判定することにより、小課題 6 に正答できる。

小課題 7 においては $63 \geq B_1 > B_2 > \dots > B_N \geq 0$ である。 $\{1, 2, \dots, 63\}$ の部分集合であってサイズ 6 以下のものは 75611761 個存在する。これらすべてを列挙し、DP で有効な解かどうか判定すると、実装にもよるが実行時間制限に収めるのは厳しいと思われる。



The 25th Japanese Olympiad in Informatics (JOI 2025/2026)

Semifinal Stage

February 1, 2026 (Shimbashi, Tokyo)

ここから 2 つの高速化の余地があり，両方を実装すると十分余裕を持って満点を得ることができる．片方のみの実装であっても満点を得ることは可能である．

- $\{1, 2, \dots, 63\}$ の部分集合であってサイズ 5 以下のもののみ列挙する (7666240 個存在する)．これで解が見つからなければ $(1, 2, 4, 8, 16, 32)$ を最適解として出力する．
- 64bit 整数を用いて，ビットシフトと論理和の演算により DP を高速化する．



4

川下り (River Rafting)

Author : 増田 拓真

小課題 1

川下りの回数は高々 N 回である。なぜなら、 N 回終了した時点で街 1 のランプの強さが N となり、すべての街を照らすようになるからである。川下りを 1 回以上 N 回以下行うとき調べるべき通り数は、川下りの順番が無視できることに注意して $\binom{2N}{N} - 1$ 通りである。 $N = 8$ のときは 12869 通りであり、これは全探索可能である。よってそれぞれの場合で実際にすべて街が照らされているかどうかをシミュレーションすればよい。時間計算量は $O(N^2 \binom{2N}{N})$ などとなる。

小課題 3

この小課題では高々 2 回の川下りによって条件が満たせる。また 2 回の川下りを行ったとしても、川下りが終わる街はどちらも同じとしてよい。

街 1 で川下りを終える場合、必要な回数は 2 回であり、そのときのコストは $2C_1$ である。街 k ($2 \leq k \leq N$) で川下りを終える場合、必要な回数は $N = 2$ のときは 1 回でコストは C_k 、 $N \geq 3$ のときは 2 回でコストは $2C_k$ である。これらの最小値をとれば答えとなる。

小課題 4

この小課題も、川下りを終了する街はすべて同じとしてよい。街 i で川下りを終えたとき、必要な回数は $N - i + 1$ 回であり、コストは $(N - i + 1)C_i$ である。これらの最小値をとれば答えとなる。

小課題 2

次のような dp を考える。

$dp[i][j][k]$ = 街 i を根とした部分木で、川下りを j 回行い、
照らされていない街の i からの深さが最大 k である場合の最小コスト



街 i の子孫の C の最小値を C_i^* とする. また街 i の子を $v_1, v_2, \dots, v_m$ とする. このとき $\text{dp}[i][j][k]$ は次の遷移で求められる.

$$\begin{aligned}\text{dp}[i][j][k+1] &= \min_{\substack{j_1+j_2+\dots+j_m=j, \\ \max(k_1, k_2, \dots, k_m) \leq k}} \left(\sum_{t=1}^m \text{dp}[v_t][j_t][k_t] \right) \\ \text{dp}[i][j][0] &= \min_j (\text{dp}[i][j'][j] + C^* \times \max(0, j - j'))\end{aligned}$$

ここで, 最終的な答えは $\min_j \text{dp}[1][j][0]$ である. dp テーブルのサイズは $\Theta(N^3)$ であり, 遷移は累積 $\min$ を用いることで $\Theta(N)$ で行えるため, 時間計算量は $\Theta(N^4)$ となる.

小課題 5, 満点

街 i の部分木で最も遠い街までの深さを h_i とする. このとき dp テーブルに必要なのは $j \leq h_i, k \leq h_i$ の部分のみである. したがって, dp テーブルはこの部分だけ計算すればよい. よって街 i_1, i_2 を子にもつ街 i の遷移にかかる時間計算量は $\Theta(h_{i_1} h_{i_2} \max(h_{i_1}, h_{i_2}))$ となる. $\Theta(h_{i_1} h_{i_2} \max(h_{i_1}, h_{i_2})) \subset O(h_{i_1} h_{i_2} N)$ から二乗の木 DP と同様の理由で全体の和をとると $O(N^3)$ であるため, この問題は $O(N^3)$ 時間で解くことができる. また最悪計算量も $\Theta(N^3)$ であることに注意する. 子が 3 つ以上の場合も 2 つずつマージしていけばよい.



5

新しい橋 (New Bridge)

Author : 米山 瑛士

小課題 1

首都を決めた後の操作は、重み付き無向グラフの最小全域木を求めるアルゴリズムとして知られる「プリム法」である。プリム法は priority queue などを用いることで一回 $O(M \log M)$ の計算量で実行することができ、これを N 通りの首都それぞれについて行うことで、計算量は $O(NM \log M)$ となる。

小課題 2

プリム法が最小全域木を求めるアルゴリズムであったことから、ある辺によって新しく到達可能になる頂点があるのであれば、その辺は最小全域木に含まれる。したがって、最小全域木に含まれない辺は無視してよい。はじめに最小全域木を求めておき、そのグラフ上で始点を N 通り変えてプリム法を実行することで、計算量は $O(N^2 \log N)$ となる。

小課題 3

s から到達可能な島の集合は、常に区間をなす。

$s < i$ のときを考えると、 s から i へ到達可能になるとときには、ある $j (\leq s)$ について、 j から i までの区間が到達可能となっている。この j の値は、 s から i までの辺の重みの最大値を m として、 s から番号の小さい方へ見て行ったらじめて m より大きな辺が現れた地点である。 i を固定して、 s を段々小さい番号にしていったときには、 j も同じように段々小さな番号になって行く。したがって、尺取り法の要領で、 $s = i - 1, i - 2, \dots, 1$ について順に j の値を求めることができる。同様の方法で $s = i + 1, i + 2, \dots, N$ についても計算することができ、計算量は $O(NQ)$ となる。



小課題 4,5, 満点解法

プリム法と並んで最小全域木を求めるアルゴリズムである，クラスカル法について考える．

クラスカル法のあるステップで，連結成分 A と連結成分 B が， A に属する頂点 x と B に属する頂点 y を結ぶ辺 (x, y) によってマージされるとする．このとき， A に属する頂点 a ， B に属する頂点 b について， $D_{a,b} = |A| + D_{y,b}$ と定まる．ここで，頂点 b の不便度 $D_{1,b} + D_{2,b} + \dots + D_{N,b}$ の式に，先程の式が表す代入操作を繰り返すことによって，最終的に $|A_1| + |A_2| + \dots$ のような形で表すことができる．（ $D_{b,b} = 0$ に注意せよ．）この式に $|A|$ が現れる回数は，最小全域木を辺 (x, y) で切ったときに x 側にある頂点の総数 k と等しいことが分かる．したがって， B に含まれる頂点 b のスコアに $|A| \times k$ を一律に加算する操作を行えばよい．同様に， A に含まれる頂点 a のスコアにも $|B| \times (N - k)$ を加算する．

グラフがパスの場合，この加算操作は区間加算に相当する．値は最後に求められれば良いため，imos 法を用いることができる．

一般のグラフの場合，この加算操作は，「マージ過程を表す木」において部分木のノード全体に加算する操作に対応する．値は最後に求められれば良いため，部分木の根の部分に値を記録しておき，それらの値を足し合わせながらトップダウンに子の頂点へ伝播させてゆけばよい．

計算量は，最小全域木を求めるための辺のソートがボトルネックとなり， $O(M \log M)$ となる．



JOI 2025/2026 セミファイナル 6問目 奇妙な機械 (Strange Machine) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- 表裏が白か黒の色をしたタイルが N 枚あります
- 表, 裏の色の情報は文字列 S, T で与えられます
- 2 つのタイルをマージして 1 つのタイルにする操作ができます

- 表裏が白か黒の色をしたタイルが N 枚あります
- 表, 裏の色の情報は文字列 S, T で与えられます
- 2つのタイルをマージして1つのタイルにする操作ができます
- タイル a, b からタイル c を作れます
- c の**表**の色は a の**裏**の色と b の**表**の色が同じなら黒
- そうでないなら白
- c の**裏**の色は a の**表**の色と b の**裏**の色が同じなら黒
- そうでないなら白

0

問題概要

- 表裏が白か黒の色をしたタイルが N 枚あります
- 表, 裏の色の情報は文字列 S, T で与えられます
- 2 つのタイルをマージして 1 つのタイルにする操作ができます
- タイル a, b からタイル c を作れます
- Q クエリ解きたいです
- 変更クエリと判定クエリがあります

0

問題概要

- Q クエリ解きたいです
- 変更クエリと判定クエリがあります
- 変更クエリ: 1 枚のタイルを別のものに変更
- 判定クエリ: 範囲 $[L, R]$ のタイルを取り出して並べる
- 先ほどの操作を繰り返して表面が白色のタイルの枚数を M 枚にできるか判定

0

問題概要

- Q クエリ解きたいです
- 変更クエリと判定クエリがあります
- 変更クエリ: 1 枚のタイルを別のものに変更
- 判定クエリ: 範囲 $[L, R]$ のタイルを取り出して並べる
- 先ほどの操作を繰り返して表面が白色のタイルの枚数を M 枚にできるか判定
- S, T のある範囲について判定するというもの

0

問題概要

- Q クエリ解きたいです
- 変更クエリと判定クエリがあります
- 変更クエリ: 1 枚のタイルを別のものに変更
- 判定クエリ: 範囲 $[L, R]$ のタイルを取り出して並べる
- 先ほどの操作を繰り返して表面が白色のタイルの枚数を M 枚にできるか判定
- S, T のある範囲について判定するというもの
- S, T のある RANGE について ...

0

問題概要

- Q クエリ解きたいです
- 変更クエリと判定クエリがあります
- 変更クエリ: 1 枚のタイルを別のものに変更
- 判定クエリ: 範囲 $[L, R]$ のタイルを取り出して並べる
- 先ほどの操作を繰り返して表面が白色のタイルの枚数を M 枚にできるか判定
- S, T のある範囲について判定するというもの
- S, T のある **RANGE** について ...

- Q クエリ解きたいです
- 変更クエリと判定クエリがあります

- 変更クエリ: 1 枚のタイルを別のものに変更
- 判定クエリ: 範囲 $[L, R]$ のタイルを取り出して並べる
- 先ほどの操作を繰り返して範囲が白色のタイルの枚数を M 枚にできるか判定

- S, T のある範囲について判定するというもの
- S, T のある RANGE について ...

- $N \leq 300\,000$
- $Q \leq 300\,000$
- タイルの色や範囲に特殊な制約はなし

0

小課題

小課題番号	N	Q	クエリ	配点
1	6			6 点
2	100		判定のみ	10 点
3	500		判定のみ	9 点
4	1700		判定のみ	8 点
5	10000	10000		23 点
6	100000	100000		14 点
7				30 点

Subtask 1

$$N \leq 6$$

1

全探索

- タイルの色は 4 通り
- 長さ n の列の状態は $4^n = 2^{2n}$ 通り
- $n \leq 6$ ならこれは 4096 以下
- 全部の状態を列挙できないか？

1

全探索

- タイルの色は 4 通り
- 長さ n の列の状態は $4^n = 2^{2n}$ 通り
- $n \leq 6$ ならこれは 4096 以下
- 全部の状態を列挙できないか？
- できる
- 頂点数 $O(4^N)$ で、辺数 $O(N 4^N)$ の有向グラフの到達判定などとみなせる

1

グラフにする

- 状態の遷移を表すグラフを作ろう
- タイルの列の状態を数字などにしたい
- 4進数 or 5進数(カードがないことを表す数字をつける)で解釈すると楽
- いずれにせよ $O(N 4^N)$ でグラフが作れる

1

グラフにする

- 状態の遷移を表すグラフを作ろう
- タイルの列の状態を数字などにしたい
- 4進数 or 5進数(カードがないことを表す数字をつける)で解釈すると楽
- いずれにせよ $O(N 4^N)$ でグラフが作れる
- 前準備で全頂点間の到達可能性を調べておく
- DFS とか BFS とか お好きな方法で

1

グラフにする

- クエリはこれを使えば $O(1)$ とかで答えられる
- 全体で $O(N^4 + Q)$ など
- 色々定数倍が軽くなると思うので実装が酷すぎなければ通ると
思われる

1

グラフにする

- クエリはこれを使えば $O(1)$ とかで答えられる
- 全体で $O(N^4 + Q)$ など
- 色々定数倍が軽くなると思うので実装が酷すぎなければ通ると思われる
- 2 ページ後にある実装テクニックを参照
- どうしても TLE するならグラフを対応する状態のタイルの枚数で層にして分割するとかがある
- 多分小課題 2 よりも実装が大変なのでやめた方がいいです

1

別解

- 判定クエリにおいて操作の仕方は $O((N - 1)!)$ 通りになる
- 全部試せばいい
- $O((N - 1)! Q)$ など解ける
- 特に、状態 s から表が白の枚数を m にしたいという組 (s, m) が 2 回以上出てきたらその後はサボるという高速化ができる
- **メモ化再帰** という名前がついています

1

実装テクニック

- これらの解法においてタイルの状態の列を持つ必要がある
- タイルの各状態を 0 から 3 の整数の 4 通りに対応したい
- でもどうやって？
- $2 * (\text{表の色が黒なら } 0, \text{ 白なら } 1) + 1 * (\text{表の色が黒なら } 0, \text{ 白なら } 1)$ という表し方が楽

1

さらに考察

- タイルの各状態を 0 から 3 の整数の 4 通りに対応したい
- $2 * (\text{表の色が黒なら } 0, \text{ 白なら } 1) + 1 * (\text{裏の色が黒なら } 0, \text{ 白なら } 1)$ という表し方が楽
- この表し方で操作を解釈する
- c の**表**の色は a の**裏**の色と b の**表**の色が同じなら黒
- そうでないなら白
- c の**裏**の色は a の**表**の色と b の**裏**の色が同じなら黒
- そうでないなら白

1

さらに考察

- タイルの各状態を 0 から 3 の整数の 4 通りに対応したい
- $2 * (\text{表の色が黒なら } 0, \text{ 白なら } 1) + 1 * (\text{裏の色が黒なら } 0, \text{ 白なら } 1)$ という表し方が楽
- この表し方で操作を解釈する
- c の**表**の色は a の**裏**の色と b の**表**の色の **XOR**
- c の**裏**の色は a の**表**の色と b の**裏**の色の **XOR**

1

さらに考察

- c の**表**の色は a の**裏**の色と b の**表**の色の **XOR**
- c の**裏**の色は a の**表**の色と b の**裏**の色の **XOR**
- XOR は排他的累積和のこと
- (この問題の制約では) 足し算して 2 で余りを取ると思って OK

1

以降の前提

- これ以降は黒を 0 に，白を 1 に対応させます
- c の**表**は a の**裏**と b の**表**の **XOR**
- c の**裏**は a の**表**と b の**裏**の **XOR**

1

以降の前提

- これ以降は黒を 0 に，白を 1 に対応させます
- 特に #-@ のように書いたら表が # で裏が @ のタイルと思ってください
- 例えば 1-0 なら表が白で裏が黒のタイルという意味
- * は 0, 1 のどちらも ok という意味
- 例えば 0-* なら表が黒で裏はなんでも OK のタイル
- 判定クエリは 1-* の数を M 枚にしてねというもの

Subtask 2

$N \leq 100$, 判定クエリのみ

2

判定クエリのみ

- ありうる全ての判定クエリについて答えを求めておこう
- $dp[i][j][k] :=$ 区間 $[i, j)$ から 1^* を k 枚にできるか？
- いわゆる区間 DP
- 遷移を考えると区間を 1 枚にする場合に困る
- これも補助的に区間 DP をしよう
- $dp'[i][j][k] :=$ 区間 $[i, j)$ から k に対応したタイル 1 枚にできるか？

2 DP

- $dp'[i][j][k] :=$ 区間 $[i, j)$ から k に対応したタイル 1 枚にできるか？
- これは $O(N^3)$ とかでできる
- $dp[i][j][k] :=$ 区間 $[i, j)$ から 1^* を k 枚にできるか？
- dp' があれば $O(N^4)$ とかでできる
- 全体で $O(N^4 + Q)$

Subtask 3

$N \leq 500$, 判定クエリのみ

3

前書き

- 小課題 2 で区間 DP を考えた
- この方針は同じ
- 高速化のアイデアとして以下の 2 つがある
- 一般的に使えるもの
- この問題固有の性質を使うもの
- 1 つ実装すると小課題 3 が解けて、2 つ実装すると小課題 4 が解けるはず
- 定数倍にもかなり抛ると思います

3

前書き

- 高速化のアイデアとして以下の 2 つがある
- 一般的に使えるもの
- この問題固有の性質を使うもの
- まずは「一般的に使えるもの」の方を書きます

3

DP の定義

- DP の定義を思い出す
- $dp'[i][j][k] :=$ 区間 $[i, j)$ から k に対応したタイル 1 枚に できるか？
- $dp[i][j][k] :=$ 区間 $[i, j)$ から 1^* を k 枚にできるか？
- いずれもできるか？ の形
- つまり true / false の bool の形

3

bitset 高速化

- こういうbool の形の DP は bitset で高速化できることが多い
- ワードサイズを w として $1/w$ がオーダーにつく
- 大体 $w = 64$ であることが多い
- bitset 高速化をすると $O(N^4/w + Q)$ になる

Subtask 4

$N \leq 1700$, 判定クエリのみ

4

前書き

- 高速化のアイデアとして以下の 2 つがある
- 一般的に使えるもの
- この問題固有の性質を使うもの
- ここでは「この問題固有の性質を使うもの」の方を書きます

4

考察

- この問題の判定クエリが解けるなら, 1^* の最大化や最小化も解けそう
- 最大化を考えてみよう！
- まず, 操作によって 1^* の数の差分はどうなる？

4

差分に着目

- この問題の判定クエリが解けるなら、 $1-*$ の最大化や最小化も解けそう
- 最大化を考えてみよう！
- まず、操作によって $1-*$ の数の差分はどうなる？
- 減る方は $-2, -1$ がありえる
- 増える方は $+1$ がありえる
- 差分は -2 から $+1$ の間をとる

- **性質 01:**
- 現在の値と最大値の間は全部取れる
- これより上の性質が成り立つことがいえる
- 補題として「差分は -2 から $+1$ の間をとる」を用いる
- 数学が得意な人は「中間値の定理」と思うとピンと来るかも

- **性質 01:**

- 現在の値と最大値の間は全部取れる

- **証明 01:**

- 補題「差分は -2 から $+1$ の間をとる」を用いる
- 現在の 1^* を x として 1^* を最大化する操作列を 1 つ取る
- 最終結果が x' になったとする ($x \leq x'$)
- $x < z < x'$ で z 枚にできないような z が存在するとする
- このとき $z - 1$ 枚以下から $z + 1$ 枚以上に 1 回の操作で変化
- これは補題に矛盾 よって示された

4

最大化

- 現在の値を求めるのは簡単（累積和とかなんでも）
- 最大値を求めよう
- まず，最初に 1^* であるタイルは触らなくて良い
- タイルの列を区間に分割して，それぞれを 1 枚のタイルにする
と捉えると， 1^* であるタイルを含む区間からは 1^* は最大 1 枚
しか得られない
- それは最初に達成されてるので，区間から消して残りで作る
ことで損をしない

4

最大化

- 現在の値を求めるのは簡単（累積和とかなんでも）
- 最大値を求めよう
- まず，最初に 1^* であるタイルは触らなくて良い
- 残りは 0^* のタイルのみからなる区間での最大化
- それぞれのタイルは $0-0$ か $0-1$

4

最大化

- 残りは 0^* のタイルのみからなる区間での最大化
- それぞれのタイルは $0-0$ か $0-1$
- $[[0-1 * n] + [0-0 * m]]$ のブロックがたくさん並んでいると
思える
- ここで, n, m は非負
- $0-0$ を左に置く操作は何も意味しないと思うとこの順番になる

4

最大化

- 残りは 0^* のタイルのみからなる区間での最大化
- それぞれのタイルは $0-0$ か $0-1$
- $[[0-1 * n] + [0-0 * m]]$ のブロックがたくさん並んでいると
思える
- ここで, n, m は非負
- 各ブロックについて最大値は $(n + \min(m, 1)) / 2$ (切り捨て)
- 左から 2 つずつ操作をしてけば達成可能

4

最大化

- $[[0-1 * n] + [0-0 * m]]$ のブロックがたくさん並んでいると
思える
- ここで, n, m は非負
- 各ブロックについて最大値は $(n + \min(m, 1)) / 2$ (切り捨て)
- 左から 2 つずつ操作をしてけば達成可能
- 実は全体の最大値も各ブロックの最大値の総和になる
- ブロックを跨ぐ操作をすると 0-0 と 0-1 で操作することになる
がこれは意味がないため

4

最大化

- $[[0-1 * n] + [0-0 * m]]$ のブロックがたくさん並んでいると
思える
- ここで, n, m は非負
- 各ブロックについて最大値は $(n + \min(m, 1)) / 2$ (切り捨て)
- 左から 2 つずつ操作をしてけば達成可能
- 実は全体の最大値も各ブロックの最大値の総和になる
- ブロックを跨ぐ操作をすると 0-0 と 0-1 で操作することになる
がこれは意味がないため
- 長さについての帰納法でも示せると思う

4

最大化

- 結局のところ，最大値は $O(N)$ で求めることができる
- 最大化はまともそう
- 最小化を考える

4

最小化へ

- 結局のところ、最大値は $O(N)$ で求めることができる
- 最大化はまともそう
- 最小化を考える
- まず先ほど書いた区間 DP テーブルを観察🔍
- すると大体 true になってる

4

最小化

- 最小化を考える 区間 DP テーブルを観察 🔍
- 大体 true になってる
- 逆にそうではないケースは true / false が交互になっている感じ
- 「そうではないケース」を観察すると 0-0 や 1-1 がほとんど
- 「そうであるケース」では 3 以上はできそう

4

最小化

- 最小化を考える 区間 DP テーブルを観察🔍
- 大体 true になってる
- 逆にそうではないケースは true / false が交互になっている感じ
- 「そうではないケース」を観察すると 0-0 や 1-1 がほとんど
- 「そうであるケース」では 3 以上はできそう
- 実は成り立つ
- 証明をしよう

4

初手に着目

- まず初手に着目する
- $1-0 + 0-* \Rightarrow 0-*$
- $1-0 + 1-* \Rightarrow 1-*$
- $0-1 + 1-* \Rightarrow 0-*$
- $0-1 + 0-* \Rightarrow 1-*$
- の操作は $1-*$ の数が 1 だけ増減する
- 上 3 つが -1, 下 1 つは +1 になる

4

初手に着目

- $1-0 + 0-* \Rightarrow 0-*$
 - $1-0 + 1-* \Rightarrow 1-*$
 - $0-1 + 1-* \Rightarrow 0-*$
 - $0-1 + 0-* \Rightarrow 1-*$
-
- これらができないとき, $0-1, 1-0$ は存在するなら右端になる
 - 操作ができないというのは操作しても保たれる
-
- このようなとき $1-*$ の数の偶奇は不変で, 減らす方は 2 ずつ減っていく
 - これらの操作のいずれかができるとしよう

4

初手に着目

- $1-0 + 0-* \Rightarrow 0-*$
 - $1-0 + 1-* \Rightarrow 1-*$
 - $0-1 + 1-* \Rightarrow 0-*$
 - $0-1 + 0-* \Rightarrow 1-*$
-
- これらができるとして, そのような操作ができる 2 枚を取り出す
 - 列は 左側 L + (その 2 枚) + 右側 R
 - 左側からの最小値を m_L , 右側からの最小値を m_R とする
 - ここで, これらは共に 1 以下 (1 枚になるまで操作すれば OK)

4

初手に着目

- これらができるとして, そのような操作ができる 2 枚を取り出す
 - 列は 左側 L + (その 2 枚) + 右側 R
 - 左側からの最小値を m_L , 右側からの最小値を m_R とする
 - ここで, これらは共に 1 以下 (1 枚になるまで操作すれば OK)
-
- 今 L , 2 枚, R に 1^* が n_L, n_M, n_R 枚あるとする

4

初手で -1 ができるとき

- 今 L , 2 枚, R に $1-*$ が n_L, n_M, n_R 枚あるとする
- (1) -1 の操作ができるとき
- 2 枚での操作をしないことで $n_L + n_M + n_R$ から $m_L + n_M + m_R$ への操作列が得られる
- 最初にすることで $n_L + n_M + n_R - 1$ から $m_L + n_M + m_R - 1$ への操作列が得られる
- $1-*$ の数の差分が -2 から -1 であることを考えると, $m_L + n_M + m_R - 1$ から $n_L + n_M + n_R$ までの全てが作れる

4 初手で -1 ができるとき

- 今 L, 2 枚, R に 1-* が n_L, n_M, n_R 枚あるとする
- (1) -1 の操作ができるとき
- 1-* の数の差分が -2 から -1 であることを考えると, $m_L + n_M + m_R - 1$ から $n_L + n_M + n_R$ までの全てが作れる
- 証明: 背理法
- $m_L + n_M + m_R - 1 \leq 1 + 2 + 1 - 1 = 3$ である

4 初手で +1 ができるとき

- 今 L, 2 枚, R に 1-* が n_L, n_M, n_R 枚あるとする
- (2) +1 の操作ができるとき
- (1) と同様に考えると, $m_L + n_M + m_R$ から $n_L + n_M + n_R$ までの全てが作れる
- 証明: 背理法
- $m_L + n_M + m_R \leq 1 + 0 + 1 = 2$ である

4 初手で -1 か $+1$ ができるとき

- 結局 -1 か $+1$ ができるなら 3 以上は作ることができる
- つまり $dp[i][j][k] :=$ 区間 $[i, j)$ から 1^* を k 枚にできるか？
- の k として考えるべき範囲は $0 \leq k \leq 2$ のみで良い！
- これにより DP の計算量を落とせる
- 全体で $O(N^3/w + Q)$ など解ける
- これ単体だと $O(N^3 + Q)$ で小課題 3 が解けます

Subtask 5

$N \leq 1\,000, Q \leq 1\,000$

5

振り返り

- 区間から作れる最大値は $O(N)$ でできる
- 減らす方は 0, 1, 2 が作れるかの判定となる
- この判定をなんとか高速化したい

5

実験

- やっぱり先ほどと同様に実験を試みる
- するといくつかルールが見えてくる
- 00 ができないというのは「 1^* が 00 枚になるような操作ができない」という意味を指します

- **性質 02:**
- 0 ができないことの必要十分条件は 0-1, 1-0 が右 2 つ以外ないことである

- **性質 02:**
- 0 ができないことの必要十分条件は 0-1, 1-0 が右 2 つ以外ないことである
- **証明 02:**
- いくつかの区間に分け, 各区間が 1 つのタイルになるがそれが全て 0-* の形になってほしい
- 右 2 枚を除けば 1-1 の枚数の偶奇のみに依存
- 左の 1-1 が偶数 $\Rightarrow$ 右 2 枚に帰着

- **性質 02:**
- 0 ができないことの必要十分条件は 0-1, 1-0 が右 2 つ以外ないことである
- **証明 02:**
- 左の 1-1 が偶数 $\Rightarrow$ 右 2 枚に帰着
- できる: $[0-0, 0-*], [0-1, *-*], [1-0, 0-*], [1-1, 1-*]$
- できない: otherwise
- 左の 1-1 が奇数 $\Rightarrow$ 1-1 と右 2 枚に帰着
- できる: $1-1 + ([0-0, 1-*], [0-1, 0-*], [1-0, *-*], [1-1, 0-*])$
- できない: otherwise

- **性質 03:**
- 1 ができないことの必要十分条件は [特性 1] (1^* が偶数) と ($1-0$, $0-1$ が右端以外ない) である

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- 長さの帰納法を用いる 長さ 1 では明らか
- まず, 上の特性が成り立つ列が一手後も成り立つことを示す
- 列は $[0-0 \text{ or } 1-1] * (L - 1) + [*-*]$ と書ける
- *-* が絡まなければ成り立つ
- *-* が絡んでも成り立つことが場合わけで示される

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- 上の特性が成り立たない列が一手後で成り立たないようにできる or 1 が作れることを示す
- 右 2 つ以外に 0-1, 1-0 があると可能なのでそうでないとする
- $[0-0 \text{ or } 1-1] * (L - 2) + [0-1 \text{ or } 1-0] + [*-*]$ と書ける
- (1) 右から 2 つめが 0-1 とする
- (1 - i) $L - 2$ の中の 1-1 が偶数とする

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- $[0-0 \text{ or } 1-1] * (L - 2) + [0-1 \text{ or } 1-0] + [*-*]$ と書ける
- (1) 右から 2 つめが 0-1 とする
- (1 - i) $L - 2$ の中の 1-1 が偶数とする
- 右端が 0-* なら最初に右 2 つを merge すれば良い
- 右端が 1-* なら $L - 2$ を全部 0-0 にすれば 1-* が作れる

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- $[0-0 \text{ or } 1-1] * (L - 2) + [0-1 \text{ or } 1-0] + [*-*]$ と書ける
- (1) 右から 2 つめが 0-1 とする
- (1 - ii) $L - 2$ の中の 1-1 が奇数とする
- 右端が 0-* なら右端以外をまとめて 1-0 を作れば良い
- 右端が 1-* なら $L - 2$ をまとめると 1-* になり, 右 2 つをまとめると 0-* となり, 可能

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- $[0-0 \text{ or } 1-1] * (L - 2) + [0-1 \text{ or } 1-0] + [*-*]$ と書ける
- (2) 右から 2 つめが 1-0 とする
- (2 - i) $L - 2$ の中の 1-1 が偶数とする
- 右端が 0-* なら右端以外をまとめて 1-0 を作れば良い
- 右端が 1-* なら全てまとめると 1-* ができる

- **性質 03:**

- 1 ができないことの必要十分条件は (1-* が偶数) と (1-0, 0-1 が右端以外ない) である

- **証明 03:**

- $[0-0 \text{ or } 1-1] * (L - 2) + [0-1 \text{ or } 1-0] + [*-*]$ と書ける
- (2) 右から 2 つめが 1-0 とする
- (2 - ii) $L - 2$ の中の 1-1 が奇数とする
- 右端が 0-* なら右 2 つをまとめれば良い
- 右端が 1-* なら右端以外を 0-1 にまとめれば良い

- **性質 03:**
- 1 ができないことの必要十分条件は $(1-* \text{ が偶数})$ と $(1-0, 0-1 \text{ が右端以外ない})$ である
- **証明 03:**
- ここまでの議論を総合すると $[0-0, 1-1] * (L - 1) + [*-*]$ を考えれば良い
- $1-*$ が奇数: 右 2 つを merge すれば良い
- $1-*$ が偶数: 仮定に矛盾
- 以上より帰納法が回る

- **性質 04:**
- 2 ができないことの必要十分条件は（作れる最大値が 2）かつ（1-* が奇数）と（0-1 が右端以外ない）である

- **性質 04:**
- 2 ができないことの必要十分条件は（作れる最大値が 2 未満）かつ（1-* が奇数）と（0-1 が右端以外ない）である
- **証明 04:**
- 最大値 2 以下については自明
- 0, 1 なら無理だし, 2 なら明らかにできる
- 最大値 3 以上として, (1-* が奇数) と (0-1 が右端以外ない) に議論を絞る

- **性質 04:**
- 2 ができないことの必要十分条件は (作れる最大値が 2 未満) かつ ($1-*$ が奇数) と (0-1 が右端以外ない) である
- **証明 04:**
- ($1-*$ が奇数) と (0-1 が右端以外ない) のとき
- $[0-0 \text{ or } 1-1] * (L - 1) + [0-1 \text{ or } 1-1]$ と書ける
- $1-*$ の偶奇は不変なので良い
- そうでないケースを考える
- 今の $1-*$ が 2 以下なら最大値に向かう途中でできる

- **性質 04:**

- 2 ができないことの必要十分条件は (作れる最大値が 2 未満) かつ (1^* が奇数) と ($0-1$ が右端以外ない) である

- **証明 04:**

- (1^* が奇数) と ($0-1$ が右端以外ない) でないケースを考える
- 今の 1^* が 2 以下なら最大値に向かう途中でできる
- 今の 1^* が 3 以上とする
- $[0-1, 0^*]$ or $[0-1, 1^*]$ が連続部分列に含まれてれば 2 が作れる
- この 2 枚を触らない操作列とこれを先に触る操作列を考えれば良い (小課題 4 の「初手に着目」と同様の考え方)

- **性質 04:**

- 2 ができないことの必要十分条件は（作れる最大値が 2 未満）かつ $(1-* \text{ が奇数})$ と $(0-1 \text{ が右端以外ない})$ である

- **証明 04:**

- $(1-* \text{ が奇数})$ と $(0-1 \text{ が右端以外ない})$ でないケースを考える
- $[0-1, 0-*]$ or $[0-1, 1-*]$ が連続部分列にないとする
- $[1-*] * ?, [0-0] * ?, [1-*] * ?, \dots, [0-1] * (0 \text{ or } 1)$ みたいなブロックの並びになる

- **性質 04:**

- 2 ができないことの必要十分条件は（作れる最大値が 2 未満）かつ（1-* が奇数）と（0-1 が右端以外ない）である

- **証明 04:**

- $[1-]* * ?$, $[0-0] * ?$, $[1-]* * ?$, ..., $[0-1] * (0 \text{ or } 1)$ みたいなブロックの並びになる
- (1) 1-0 がないケース
- 1-* が 1-1 のみとなり, 1-* の偶奇が不変量となるため成立
- (2) 1-0 があるケース
- 0-1 があるかないかで場合分け

- **性質 04:**

- 2 ができないことの必要十分条件は (作れる最大値が 2 未満) かつ (1^* が奇数) と ($0-1$ が右端以外ない) である

- **証明 04:**

- $[1^*]^* ?$, $[0-0]^* ?$, $[1^*]^* ?$, ..., $[0-1]^* (0 \text{ or } 1)$ みたいなブロックの並びになる
- (2) $1-0$ があるケース
- (2 - i) $0-1$ がある場合
- $1-0$ が右端以外にあれば 2 が作れ, $0-1$ が右端より良い

- **性質 04:**

- 2 ができないことの必要十分条件は (作れる最大値が 2 未満) かつ (1-* が奇数) と (0-1 が右端以外ない) である

- **証明 04:**

- $[1-*] * ?$, $[0-0] * ?$, $[1-*] * ?$, ..., $[0-1] * (0 \text{ or } 1)$ みたいなブロックの並びになる
- (2) 1-0 があるケース
- (2 - ii) 0-1 がない場合
- 1-0 が右端のみのケースを考えると $[0-0 \text{ or } 1-1] * (L - 1) + [1-0]$ となり, 1-* の偶奇が不変量となるため成立

5

解法

- 0-1, 1-0 は存在するなら右端かの判定も $O(N)$ でできる
- 0, 1, 2 ができるかは $O(N)$ で判定できる
- 最大値も $O(N)$ で判定できる
- よってクエリあたり $O(N)$ で解ける
- 全体で $O(NQ)$ となり解ける

Subtask 7

追加制約なし

7

振り返り

- 定数倍が悪いか遅い言語だと小課題 6 までだと思います
- 基本的には小課題 7 ということで, これを説明
- 小課題 5 の解法を振り返る
- 0-1, 1-0 は存在するなら右端かの判定も $O(N)$ でできる
- 0, 1, 2 ができるかは $O(N)$ で判定できる
- 最大値も $O(N)$ で判定できる

7 Segment Tree へ

- 0-1, 1-0 は存在するなら右端かの判定も $O(N)$ でできる
- 0, 1, 2 ができるかは $O(N)$ で判定できる
- 最大値も $O(N)$ で判定できる
- とあったが, 実はこれらは Segment Tree に乗る

7

解法

- 0-1, 1-0 は存在するなら右端かの判定
- これは 0-0, 0-1, 1-0, 1-1 が区間に何個あるか分かれば良い
- 一点更新 / 区間和 の Segment Tree になる
- 4 本持ってもいいし, モノイドに 4 つの個数の情報を載せても良い
- 4 つの情報を持たせるときは array を使うと定数倍が良くなる
- 0, 1, 2 ができるかの判定もこの Segment Tree でできる

7

解法

- 最大値の計算
- ブロックごとに計算できるので、ブロックの情報を持たせたい
- マージ (ACL の op) を考えると、次のような情報が欲しい
 - 左端を含むブロックの情報
 - 右端を含むブロックの情報
 - 今の区間のブロックがちょうど 1 つであるか

7

解法

- 最大値の計算
- ブロックごとに計算できるので、ブロックの情報を持たせたい
- マージ (ACL の op) を考えると、次のような情報が欲しい
 - 左端を含むブロックの情報
 - 右端を含むブロックの情報
 - 今の区間のブロックがちょうど 1 つであるか
- 後はこれらで頑張るとマージできる (実装は重いです)

7

解法

- 結局 Segment Tree で全て処理できる形になった
- 2 種類あるので，抽象化しておくとお装が楽になる
- AtCoder 環境では ACL が使えるので楽
- (抽象化できなくとも)ソラ書きできるとファイナルでは役に立ちます

7

解法

- 0-1, 1-0 は存在するなら右端か
 - 0, 1, 2 ができるか
 - 最大値
-
- これらは Segment Tree に乗る
 - $O(\log N)$ で一点変更や区間積が計算できる
-
- 全体で $O(N + Q \log N)$ で解ける

7

解法

- 0-1, 1-0 は存在するなら右端か
- 0, 1, 2 ができるか
- 最大値
- これらは Segment Tree に乗る
- $O(\log N)$ で一点変更や区間積が計算できる
- 全体で $O(N + Q \log N)$ で解ける
- ちなみに, 0-1, 1-0 が右端以外にある場合, 1, 2 は必ずできるので実装上は 0 のみで良い

Step 8

得点分布

8

得点分布

- 理想:

点数	1	2	3	4	5	6	人数	累計
100	0	0	0	0	0	0	170	170

- 選抜という意味では差がつかないな

8

得点分布

- 現実:

点数	1	2	3	4	5	6	人数	累計
100	0	0	0	0	0	0	1	1
25	0	0	0	-	-	-	1	2
10	-	0	-	-	-	-	3	5
6	0	-	-	-	-	-	9	14
0	-	-	-	-	-	-	156	170